ЛБ1

Назва теми проєкту: Керування ПК послідовностями мікрожестів.

1. АКТУАЛЬНІСТЬ ТЕМИ

Безконтактний ввід уже входить у щоденність. У лікарнях і зонах харчування важливо не торкатися зайвих поверхонь. Це зберігає чистоту і зменшує ризики.

Провідні платформи показали, що це працює. У visionOS та Meta Quest є мікрожести на кшталт тапу і свайпу великим пальцем. Вони дають точне керування і не перевантажують рухи.

Сьогодні легко зібрати прототип. Готові бібліотеки комп’ютерного зору визначають ключові точки кисті в реальному часі і працюють на звичайних ПК.

Залишається головний виклик. Це випадкові спрацювання, відомі як «Midas touch». Їх потрібно мінімізувати за допомогою послідовностей з клатчем і явним підтвердженням наміру.

Тема має прикладну і суспільну цінність. Менше дотиків означає більше гігієни та безпеки у чутливих середовищах. Користувач отримує зручний і чистий спосіб керування комп’ютером у щоденних задачах.

2. ОГЛЯД АНАЛОГІВ ТА ІСНУЮЧИХ РІШЕНЬ

На сьогодні вже є багато рішень для трекінгу рук і жестів. Роздивимося на якому залізі вони працюють, чи підтримують жести, який користувацький досвід дають і наскільки це придатне для ПК. Щоб бачити картину в цілому, беремо як програмні рішення на звичайну камеру, так і спеціалізовані сенсори та можливості XR-платформ. Зведені характеристики наведено в таблиці 1.

Таблиця 1 – Існуючі рішення та аналоги

Рішення	Платформа / залізо	Що вміє	Плюси	Мінуси
MediaPipe	ПЗ (ПК та телефон), вебкамера	Розпізнавання жестів рук	Безкоштовно, кросплатформено; легко стартувати	Жести треба навчати під задачу
Ultraleap Leap Motion	ІЧ-камера для ПК	Високоточний трекінг кисті та пальців	Стабільний трекінг, готові SDK	Додаткове залізо; робоча зона обмежена
Meta Quest Hand Tracking	XR-гарнітура	Клік, прокрутка, мікрожести	Демонструє «низькокалорійні» жести	Не ПК-периферія; залежить від екосистеми Quest
Apple Vision	visionOS	Погляд, клік, перетягування, прокрутка	Еталонні патерни проти випадкових спрацьовувань	Не є рішенням для ПК, але корисні принципи
Google Soli	Радар	Тонкі рухи без камери	Працює в темряві та без видимого огляду	Немає масового ПК-заліза;

3. МЕТА І ЗАВДАННЯ ДОСЛІДЖЕННЯ

Метою роботи є реалізувати програмне забезпечення для роботи із комп’ютером за допомоги жестів та перевірити, чи послідовності жестів забезпечують не гірший час і зручність, ніж миша та гарячі клавіші, і менше помилкових спрацьовувань, ніж одиночні жести.

Для досягнення мети мають бути вирішені задачі:

– спроєктувати словник послідовностей;
– зібрати прототип із трекінгом рук та класифікатор послідовностей;
– впровадити анти-помилкові «ворота»;
– спроєктувати експеримент та провести порівняння.

4. ОБ’ЄКТ ТА ПРЕДМЕТДОСЛІДЖЕННЯ

Об’єктом дослідження є система взаємодії з ПК за допомогою жестів рук. Використовується камерний трекінг. Система містить захоплення відео, виявлення кисті, основні точки, інтерпретацію жестів, машину, відправку подій у ОС та екранні підказки.

Ппредметом є вплив структури послідовностей жестів на ефективність взаємодії. Оцінюємо час виконання команди, точність, частоту хибних спрацьовувань. Додатково вивчаємо ефект SIMT прискорення на затримку системи. У нашому випадку це препроцесинг кадру, детекція долоні, лендмарки та обчислення ознак.

5. ОЧІКУВАНА НОВИЗНА

Очікувана новизна полягає у введенні для ПК окремого каналу вводу на основі жестів з послідовностями. Ми описуємо жести як просту граматику. Це не просто набір поз. Це узгоджена мова керування, яку легко читати, конфігурувати і вчити. Для опису мапінгів пропонується коротка нотація з профілями для

правші та шульги. Вона придатна для звичайної вебкамери і не вимагає спеціального заліза.

Новизна також у поділі розпізнавання і наміру. Рухи перетворюються на токени у часових вікнах. Далі працює обробка, яка приймає рішення окремо від сирих ознак. Це спрощує налагодження, дає прозорі правила і дозволяє швидко додавати нові патерни без зміни детектора.

Окремий внесок робить інтерфейс з живими підказками. Він може показувати стан, активні канали і останню команду та інше. Користувач розуміє, що система «бачить» і який крок очікується далі. Це зменшує криву навчання і помилки від непорозуміння.

Запропонована реалізація орієнтована на масовий паралелізм. Вона відповідає моделі SIMT. Препроцесинг кадру відображається на сітку потоків з одним потоком на піксель або на плитку з використанням спільної пам’яті. Основні точки відображаються пакетами на GPU через CUDA провайдер у ONNX Runtime. Ознаки у ковзних вікнах рахуються паралельно: один потік на одне вікно або на одну руку. Така побудова дає низьку затримку та стабільний FPS без зміни зовнішньої поведінки.

6. ПРАКТИЧНЕ ЗНАЧЕННЯ ОЧІКУВАНИХ РЕЗУЛЬТАТІВ

Практичне значення у тому, що з’являється реальний спосіб керувати ПК без дотику там, де це справді потрібно. У клініці лікар відкриває знімок і гортає історію без зняття рукавичок. На харчовому виробництві оператор переключає інструкції, не торкаючись панелі. На складі або в майстерні працівник у рукавицях підтверджує кроки і не бруднить екран.

Користувач може виконувати типові дії на ПК без миші та клавіатури. Це підвищує доступність і зменшує бар’єри. У щоденних задачах зростає швидкість і стабільність роботи.

Впровадження не потребує спеціального заліза. Досить звичайної вебкамери і ПК. Система працює на CPU та може прискорюватися на GPU. Інтеграція прозора, бо вихід це стандартні події клавіатури і миші. Налаштування робиться у конфігураторі. Навчання коротке завдяки підказкам на екрані.

7. АПАРАТНА МОДЕЛЬ ОБЧИСЛЮВАЧА ІЗ МАСОВИМ ПАРАЛЕЛІЗМОМ

Вендор. NVIDIA.

Сімейство ГПУ. GeForce 16 Series.

Модель кристала. TU117.

Архітектура ГПУ. Turing, SIMT.

Кількість TPC: 7.

Кількість SM: 14.

Кількість SP: 896 CUDA-ядер.

Пам’ять: 4 ГБ GDDR5.

Шина: 128 біт.

Пропускна здатність: 128 ГБ/с

ЛБ2

1. ТЕХНОЛОГІЧНА БАЗА (ОГЛЯД ІНСТРУМЕНТІВ, БІБЛІОТЕК І МОВ)

У проєкті базовою мовою взято Python 3. Він швидше та простіше працює з потрібними бібліотеками (OpenCV, MediaPipe, ONNX Runtime, PySide6), бо має зрілі офіційні обгортки та великий вибір готових прикладів. Це зменшує час на інтеграцію і дозволяє зосередитись на логіці мікрожестів та послідовностей. Критичні місця, де важлива мінімальна затримка, за потреби можна винести на GPU без зміни загальної архітектури.

Для комп'ютерного зору обрано OpenCV для роботи з камерою, перетворенням кольорів та базовим препроцесингом кадрів.

Для відстеження кисті використаємо MediaPipe Hands як швидку точку входу до 21 орієнтиру в реальному часі. Якщо буде потреба у власних моделях або альтернативних пайплайнах, передбачимо формат ONNX: інференс через ONNX Runtime з увімкненим GPU-бекендом (CUDA) на NVIDIA, із можливістю падати на CPU без змін коду.

Графічний інтерфейс та екранні підказки можна робити на PySide6/Qt. Це дозволить показувати індикатори режимів, стани команд та прості налаштування чутливості в одному вікні.

Інтеграцію з операційною системою на Windows буде реалізовано через стандартні виклики надсилання подій (гарячі клавіші, кліки, рух курсора).

2. ТЕОРЕТИЧНЕ ПОЯСНЕННЯ ЗАПРОПОНОВАНОГО РІШЕННЯ

2.1. Послідовний вигляд

У послідовному вигляді система працює як лінійний конвеєр «кадр – лендмарки – інтерпретація – команда». На вхід надходить відеопотік із камери, кадр приводиться до потрібного формату, після чого визначаються долоні та

орієнтир кисті. Отримані координати нормалізуються, на їх основі обчислюються ознаки рухів. Далі виконується інтерпретація жестів. За умови валідної послідовності формується команда для операційної системи, а на екрані оновлюються підказки та стани. Контекстну структуру рішення відображає IDEF0-діаграма, що наведено на рисунку 1.

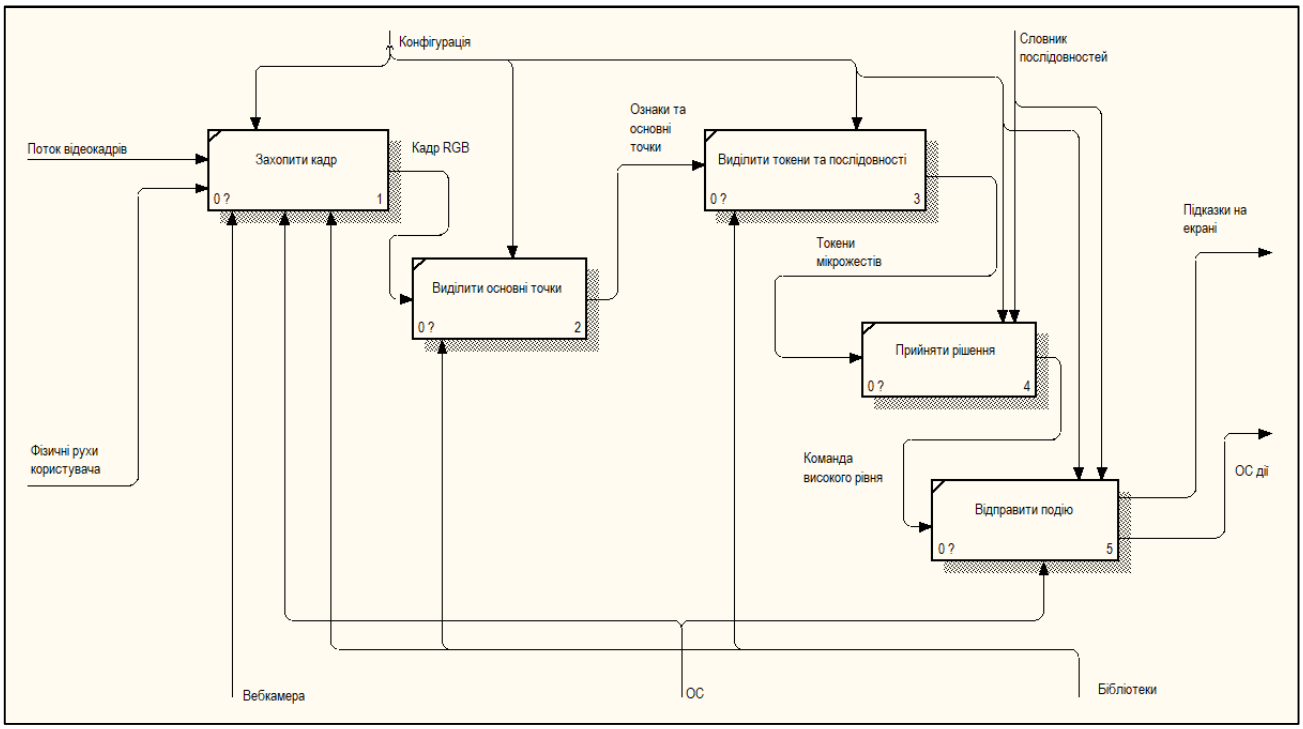

Рисунок 1 – Контекстна діаграма IDEF0

Внутрішня обробка подається лінійним ланцюгом від A1 до A5.

Блок A1 «Захопити кадр» приймає поток відеокадрів сцену через механізм вебкамеру і видає кадр у форматі RGB.

Блок A2 «Виділити основні точки» отримує цей кадр і визначає долоню та 21 орієнтир кисті для кожної знайденої руки разом із оцінкою впевненості.

Блок A3 «Виділити токени та послідовності» перетворює точки на ознаки рухів та виділяє базові дії.

Блок A4 «Прийняти рішення» застосовує словник послідовностей і правила підтвердження наміру. На виході формується команда високого рівня з урахуванням активного вікна ОС.

Блок A5 «Відправити подію» перетворює цю команду на конкретні дії операційної системи (комбінації клавіш, клацання, переміщення курсора), оновлює екранні підказки.

2.2. Паралельний вигляд

Паралельний варіант організовано як конвеєр з чотирьох ярусів. Захоплення кадру, виявлення лодонь, ухвалення рішення та інтеграція з ОС. Між ярусами стоять кільцеві буфери і черги. Вони дають незалежність обчислень та просту синхронізацію «виробник-споживач».

Ярус захвату читає кадри з камери і кладе їх у кільцевий буфер кадрів. Ярус виявлення лодонь бере останній кадр з буфера, виконує препроцесинг, детекцію долоні, обчислення точок та формує ознаки. Результат потрапляє у чергу дій.

Ярус команд читає з цієї черги, обробляє дії, формує намір та кладе команду у чергу команд.

Ярус UI OS витягує команди, надсилає події у систему і оновлює підказки на екрані.

Уся схема зображена на рисунку 2.

Рисунок 2 – Паралельно ярусна модель.

Обчислення у ярусі виявлення лодонь планується виконувати на GPU у моделі SIMT. Це скорочує час препроцесингу, детекції та точок.

ЛБ3-4

1. ОПИС МЕТОДОЛОГІЇ ПРОВЕДЕННЯ ЕКСПЕРИМЕНТУ

Метою є дослідження впливу обчислювального бекенду й схеми жестів на затримку, частоту хибних активацій та швидкість виконання користувацьких сценаріїв.

Досліджувані залежності.

- Бекенд інференсу: CPU (TFLite/XNNPACK) проти GPU (ONNX Runtime CUDA 12.6).
- Роздільна здатність камери: 320x180, 1280x720, 1920x1080.

Метрики оцінки:

- FPS_avg (кадри/с).
- false_activations – хибні активації.
- task_time_ms – час виконання сценарію.
- task_success (%) – успішність сценаріїв.

Експеримент необхідно провести для декількох основних комбінації.

Сценарії:

- Сценарій 1. Створити та відкрити папку.
- Сценарій 2. Відкрити браузер, ввести у пошуковий рядок «Hello world», відкрити сторінку Вікіпедії.
- Сценарій 3. Скопіювати файл з одної папки у іншу.

Очікувані

- Кількість кадрів не менша 15.
- Помилки не є руйнівними.
- Кількість помилок не перевищує 3.
- Час виконання завдання ніж клавіатура + миша не довша ніж у 3 рази.

2. АКУМУЛЯЦІЯ РЕЗУЛЬТАТІВ (ТАБЛИЦЯ 1)

Бекенд	Розмір	FPS_avg	Кількість помилок	Сценарій	Тип взаємодії	Витрачений час, мс
cpu	1280 x 720	15,6	1	1	Жести	48692,934
cpu	1280 x 720	15,6	0	1	Миша + клавіатура	15127,082
cpu	1280 x 720	15,6	0	2	Жести	65129,009
cpu	1280 x 720	15,6	0	2	Миша + клавіатура	16206,573
cpu	1280 x 720	15,6	3	3	Жести	80317,976
cpu	1280 x 720	15,6	0	3	Миша + клавіатура	13401,183
cpu	320 x 180	22,2	5	2	Жести	80433,011
cpu	1920 x 1080	14,3	2	2	Жести	59379,759
cpu	1920 x 1080	14,3	6	3	Жести	58862,586

3. АНАЛІЗ РОБОТИ

У ході роботи було реалізовано та експериментально перевірено прототип системи керування комп’ютером за допомогою жестів рук, що підтримує послідовності жестів і підтвердження команд.

Попередні вимірювання показали, що при активації GPU-бекенду (ONNX Runtime з CUDA) система здатна працювати з прийнятною частотою кадрів та затримкою, однак у поточній реалізації прототип залишається нестабільним: періодично виникають проблеми із завантаженням моделей, коректною обробкою формату тензорів та узгодженістю роздільної здатності входу. Це не дозволило зібрати повноцінний масив статистичних даних для строгого кількісного порівняння CPU та GPU-режимів, але якісні спостереження підтверджують очікуваний напрямок: при коректній роботі GPU-шляху затримка від кадру до відправки гарячої клавіши суб’єктивно сприймається як менша, а відеопотік – більш плавним.

Порівняння «жести проти миші+клавіатури» показало, що у поточному стані система не є повноцінною заміною традиційним засобам введення. Для базових сценаріїв (перемикання вікон, прості комбінації гарячих клавіш) жести дозволяють виконати задачу, але зазвичай повільніше, ніж досвідчений користувач мишею та клавіатурою. Водночас система виявляється перспективною саме для специфічних режимів роботи, де дотик до поверхонь небажаний або фізично незручний: стерильні зони (медицина, операційні, маніпуляційні), харчове виробництво, лабораторії, тощо. У таких умовах навіть повільніше, але «безконтактне» керування може бути виправданим.

Додатковий словник жестів і підтримка послідовностей дозволяють відображати на жести складніші команди й макроси (наприклад, послідовність дій у віконному менеджері чи запуск кількох програм поспіль). Експерименти з бімануальним підтвердженням показали, що така схема суттєво зменшує кількість хибних активацій, але збільшує час взаємодії: користувач змушений усвідомлено підтверджувати кожну «дорогу» команду другою рукою. З огляду на дослідницький статус проєкту, це виглядає прийнятним компромісом між швидкістю та надійністю.

Окремо варто відзначити обмеження поточного підходу: система працює через емуляцію стандартних гарячих клавіш ОС, тобто поводиться як «розумний пульт до Windows», а не як повністю інтегрований інтерфейс. Це значно обмежує гнучкість. Теоретично можливим є більш глибоке підключення до прикладних програм (через API, протоколи або внутрішні події), що дозволило б формувати високорівневі команди без проміжного перетворення в загальносистемні гарячі клавіші.